

CPSC 533V: Learning to Move

(Reinforcement Learning)

Michiel van de Panne

Department of Computer Science
The University of British Columbia



Registration

- we're looking for a larger room
- 27 registered, 10+ looking to register
- lectures will be recorded until at least drop deadline, links posted on Piazza -> lectures

Lecture Plan Wed Sept 4, 2024

- what is special about movement?
- course administrivia
- what is RL?
 - what can it solve? how does it differ from other ML problems?
- why Deep RL, and some caveats
- icebreaker

Movement – what is so special?

- Q: Why do we have a brain?
- A: To produce adaptable and complex movements [Wolpert]
- Movement is the only* way we have of:
 - interacting with the world
 - communicating: speech, gestures, writing
- “motor control chauvinism”
 - chau·vin·ism [shoh-vuh-niz-uhm]
 - 2. biased devotion to any group, attitude, or cause
- “athletic intelligence”

Why do we have a brain?



- because computers were not yet invented
- to behave

- example: sea squirt

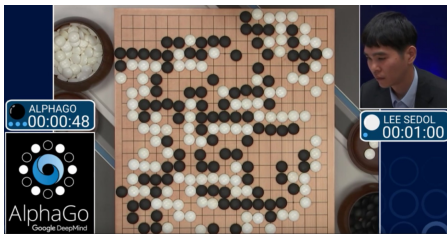


- larval stage: primitive brain & eye, swims around, attaches to a rock
- adult stage: sits. digests brain.

Example motions

- (videos)

What is RL?

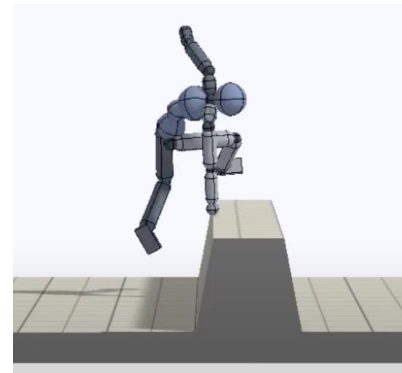


[AlphaGo]



Wayve's autonomous cars 'learning' to drive in London...
trafficttechnologytoday.com

[Wayve]



[DeepMimic]



[Anymal, ETHZ]

The lecture materials are based in part on, or inspired by, slides/blogs/demos from:

- Rich Sutton, U Alberta [Sutton] <http://incompleteideas.net/book/the-book-2nd.html>
- Emma Brunskill, Stanford [Brunskill] <https://web.stanford.edu/class/cs234/index.html>
- David Silver, UCL [Silver] <http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching.html>
- Xue Bin Peng, SFU [Peng]
- Pieter Abbeel, UC Berkeley [Abbeel]
- Sergey Levine, Chelsea Finn, John Schulman, UC Berkeley [UC Berkeley]
- Mark Schmidt, UBC [Schmidt]
- Andrej Karpathy [Karpathy] <https://karpathy.github.io/2016/05/31/rl/>
- Lillian Weng [Weng] <https://lilianweng.github.io/lil-log/2018/04/08/policy-gradient-algorithms.html>

Learn to make good sequences of decisions

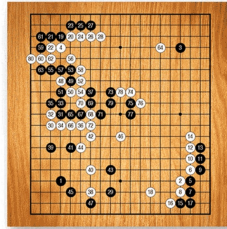
[Brunskill]

- Drive a car
- Defeat the world champion at Go
- Manage an investment portfolio
- Sequence a series of medical tests and interventions
- Control a power station or a chemical process to maximize revenue
- Make a humanoid robot walk
- Direct attention for a computer vision task
- Decide on the next medical investigation or treatment

Defining good decisions: rewards

- Fly stunt manoeuvres in a helicopter
 - +ve reward for following desired trajectory
 - -ve reward for crashing
 - Defeat the world champion at Backgammon
 - +/-ve reward for winning/losing a game
 - Manage an investment portfolio
 - +ve reward for each \$ in bank
 - Control a power station
 - +ve reward for producing power
 - -ve reward for exceeding safety thresholds
 - Make a humanoid robot walk
 - +ve reward for forward motion
 - -ve reward for falling over
 - Play many different Atari games better than humans
 - +/-ve reward for increasing/decreasing score
- [Abbeel]

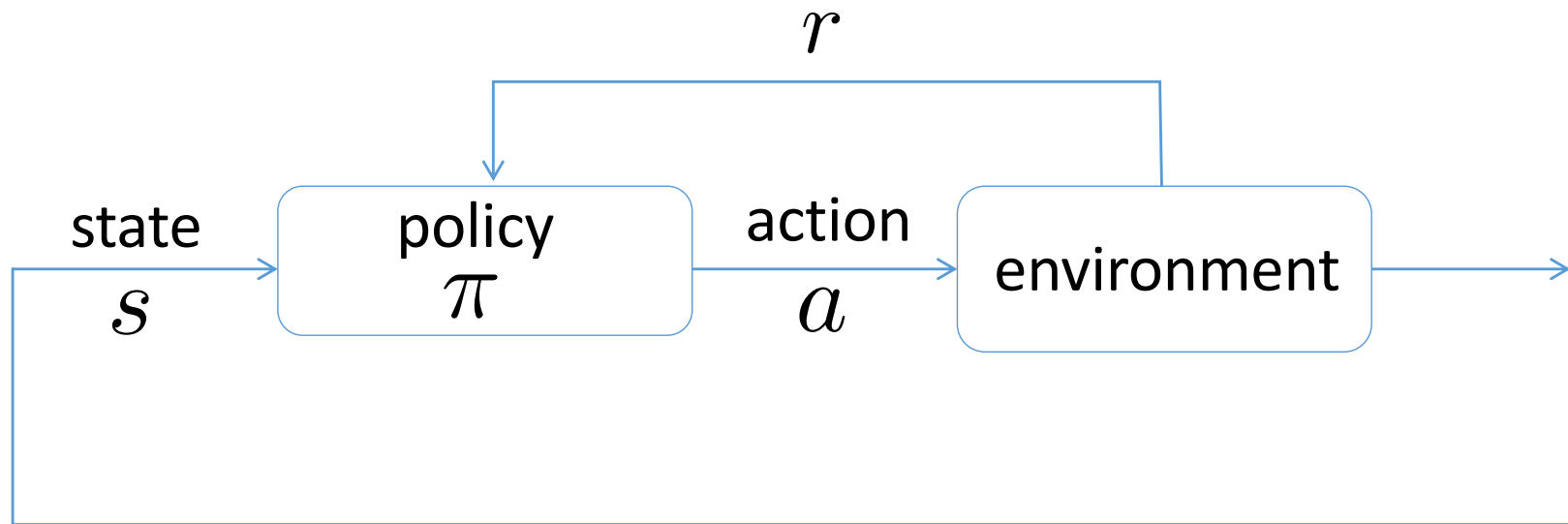
RL involves:



- optimization
 - find an optimal way to make sequential decisions
- delayed consequences
 - decisions now can impact things much later, e.g., climate change
 - challenge: temporal credit assignment is hard (what caused later high or low rewards?)
 - challenge: need to reason about long-term ramifications
- exploration
 - explore vs exploit tradeoff: try a new restaurant or go to one you already like?
 - current policy impacts future data collection: potential instabilities
- generalization
 - impossible to visit all states during learning, e.g. image input to a policy

~[Brunskill]

Reinforcement Learning



maximize sum of rewards: $G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$

$$0 \leq \gamma \leq 1$$

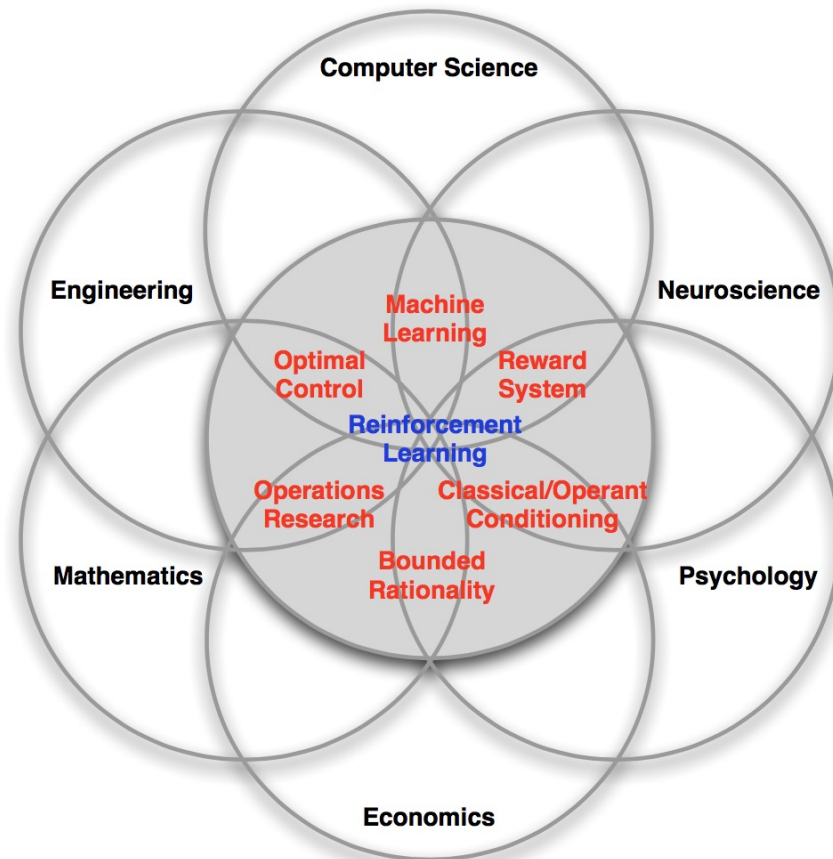
Discrete vs Continuous

- discrete states, discrete actions
 - e.g., Go, Chess, tic-tac-toe
 - “tabular” policies for small problems
- continuous states, discrete actions
 - e.g., Atari games, DOTA, cart-and-pole
- continuous states, continuous actions
 - robotics, process control, autonomous driving
- discrete states, continuous actions – less common?

RL (and ML) "Word Salad"

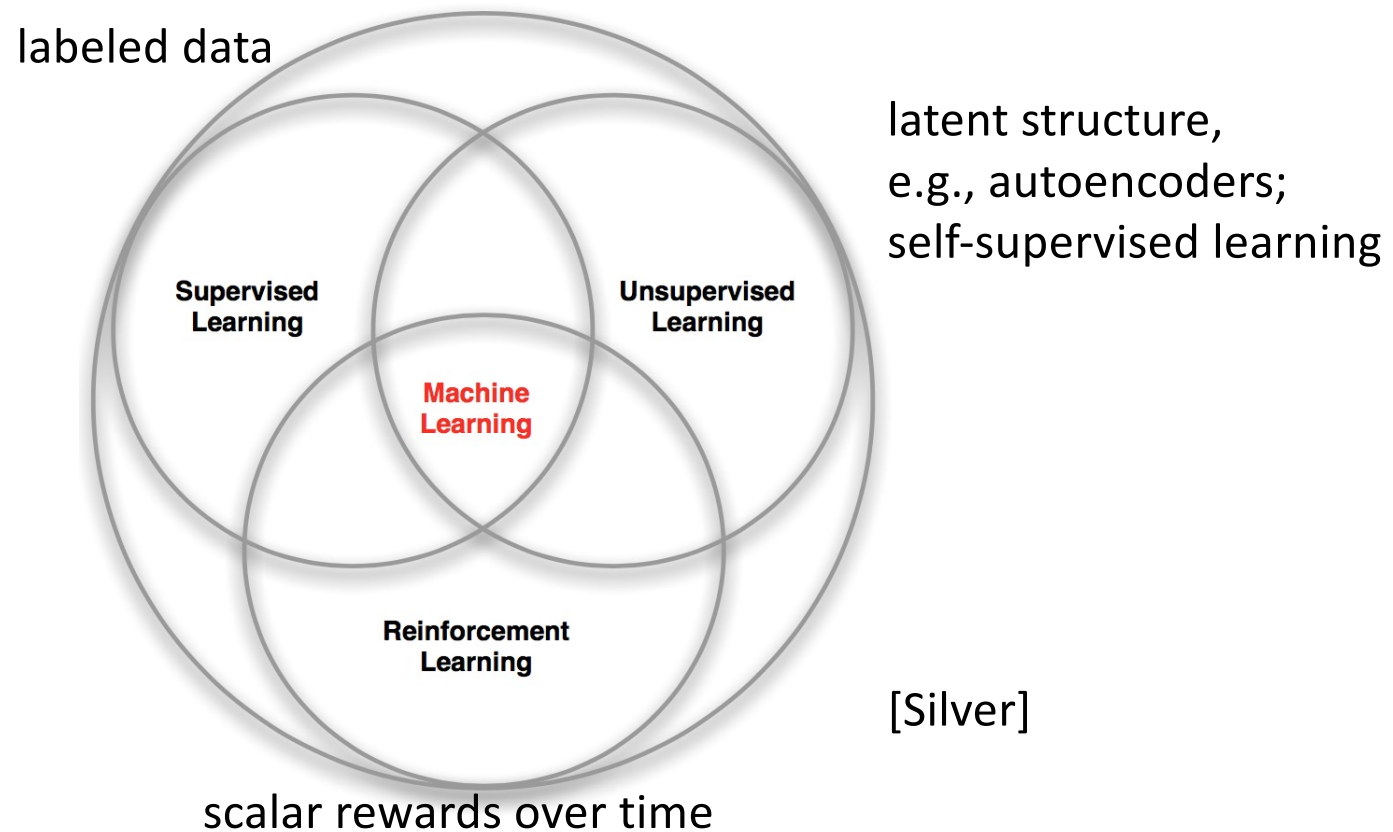
behavior cloning hindsight replay early termination
MDP LLMs
actor MPC dynamic programming
foundation model entropy bonus state-action value fn transfer learning
imitation learning inverse RL DDPM On policy vs Off policy
value fn critic options experience replay POMDP DAGGER Online vs Offline
DDPG decision transformer GAN bandits aleatoric vs epistemic uncertainty
bootstrapping TRPO, PPO policy gradient deadly triad

The Many Faces of Reinforcement Learning



[Silver]

Branches of Machine Learning

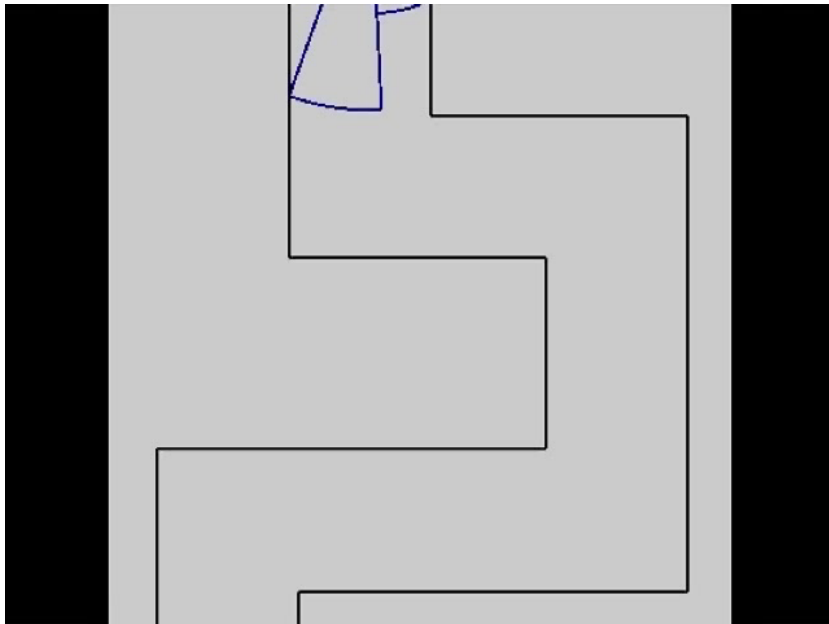


RL is the OG...

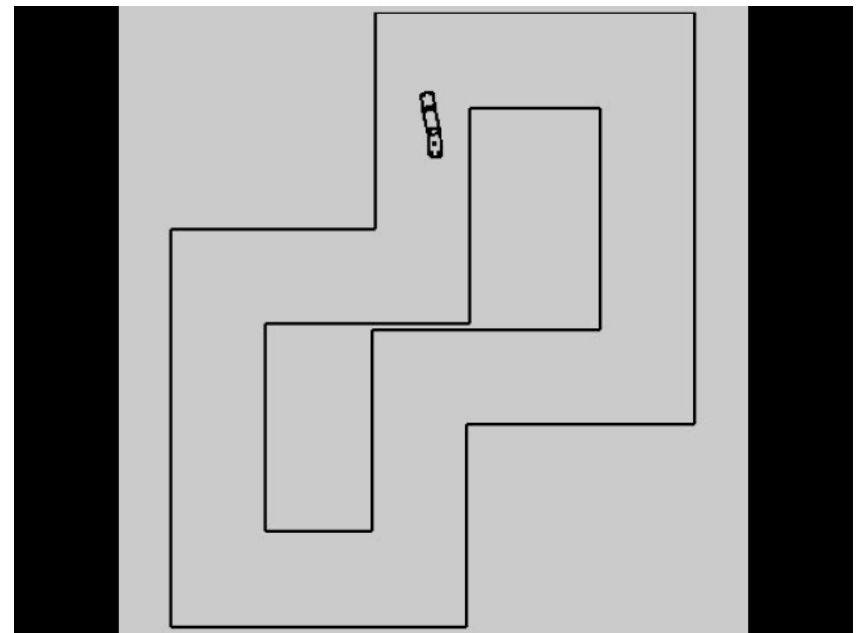
- “In the beginning, ML was RL” – Andrew Barto
 - combines SEARCH and MEMORY
 - search: trial-and-error (data collection)
 - memory: cache solutions, i.e., policy, value-function

Simple Example: Backing up a Truck

$\pi : \mathbb{R}^5 \rightarrow \mathbb{R}$ input: 4 distances + cab-angle
output: steering angle



backing up a double trailer

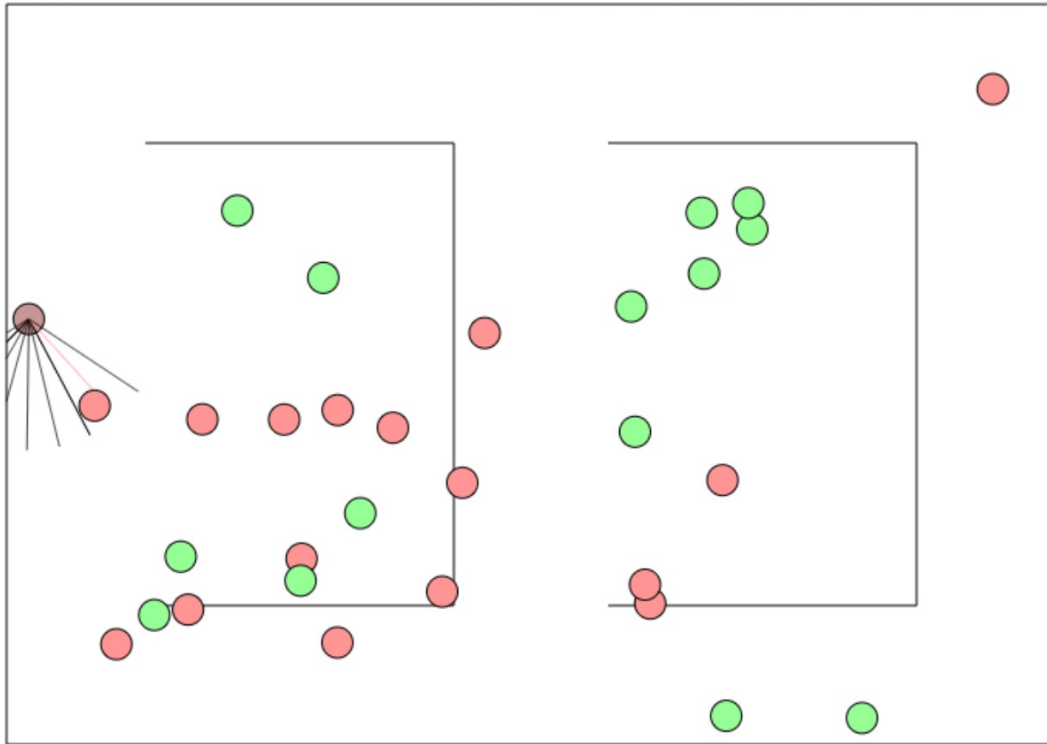


[“Learning to Steer on Winding Tracks Using Semi-Parametric Control Policies”, ICRA 2005]

Simple Example

state: 9 sensors x 3 values x 2 timesteps

actions: move in 5 directions



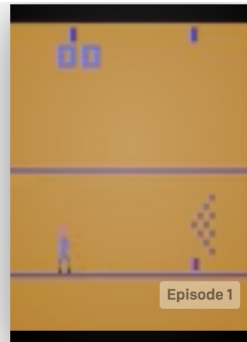
[<https://cs.stanford.edu/people/karpathy/convnetjs/demo/rldemo.html>]

OpenAI Gym

[<https://openai.com/research/openai-gym-beta>]



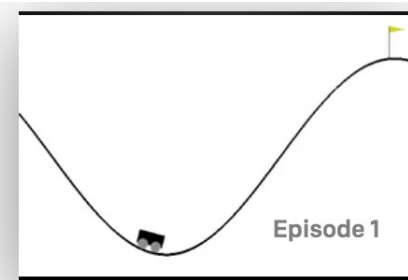
Berzerk-v0



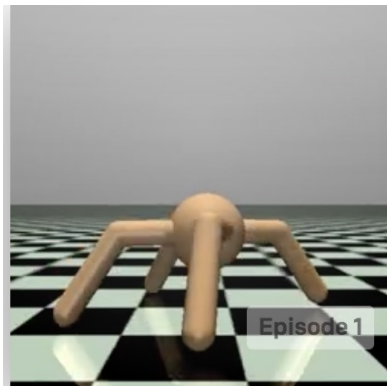
Bowling-ram-v0



CartPole-v1



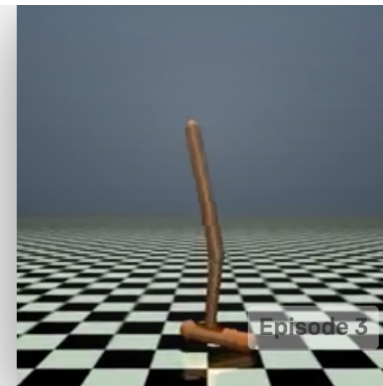
MountainCar-v0



Ant-v2



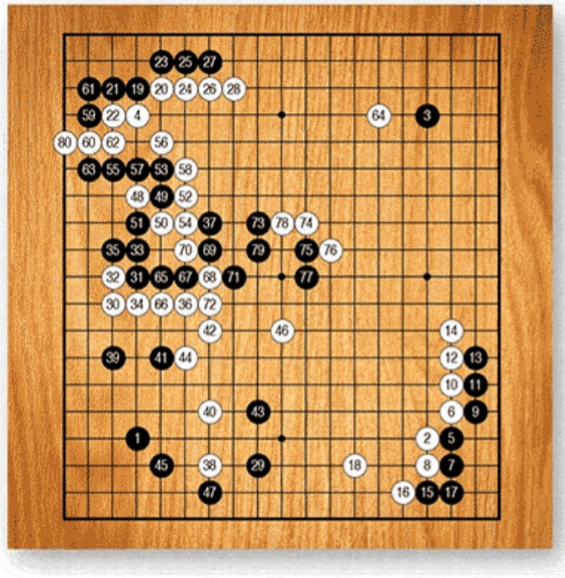
HalfCheetah-v2



Hopper-v2

Why Deep RL?

AlphaZero (DeepMind)

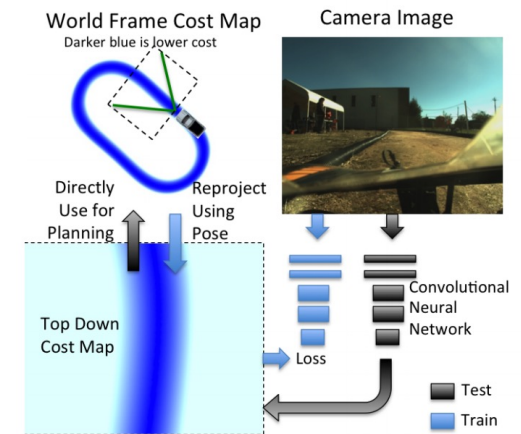


[DeepMind: Nature 2017]

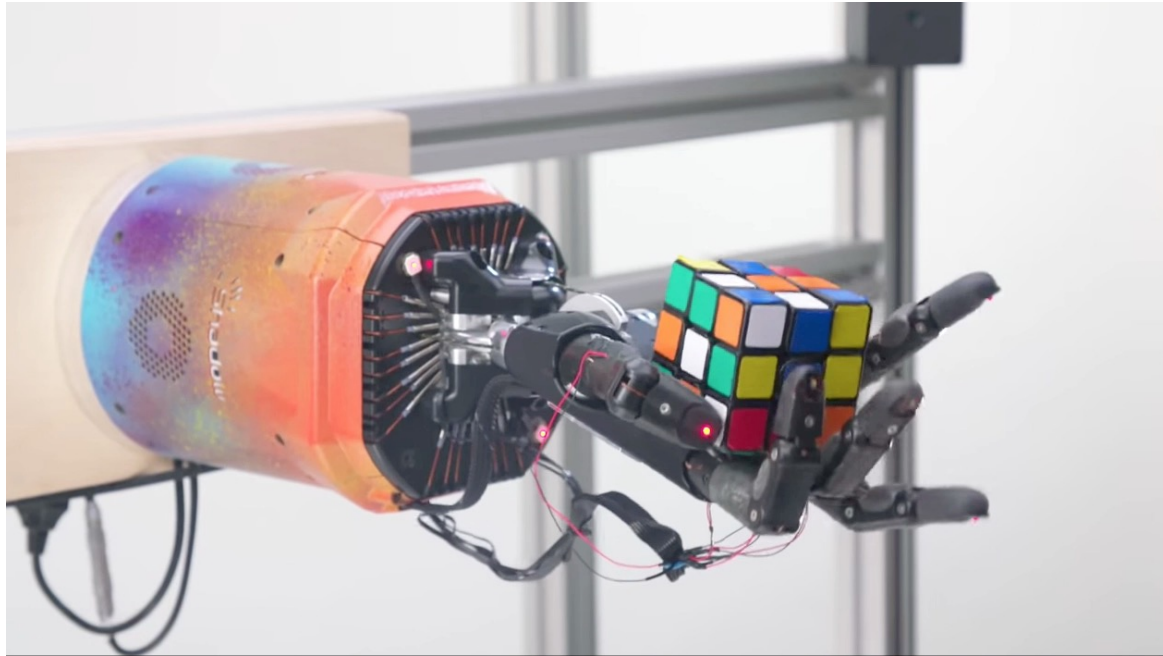
Aggressive autonomous driving of a scale vehicle

(Georgia Tech)

[Georgia Tech: CoRL 2017]



Rubik's cube manipulation (OpenAI)



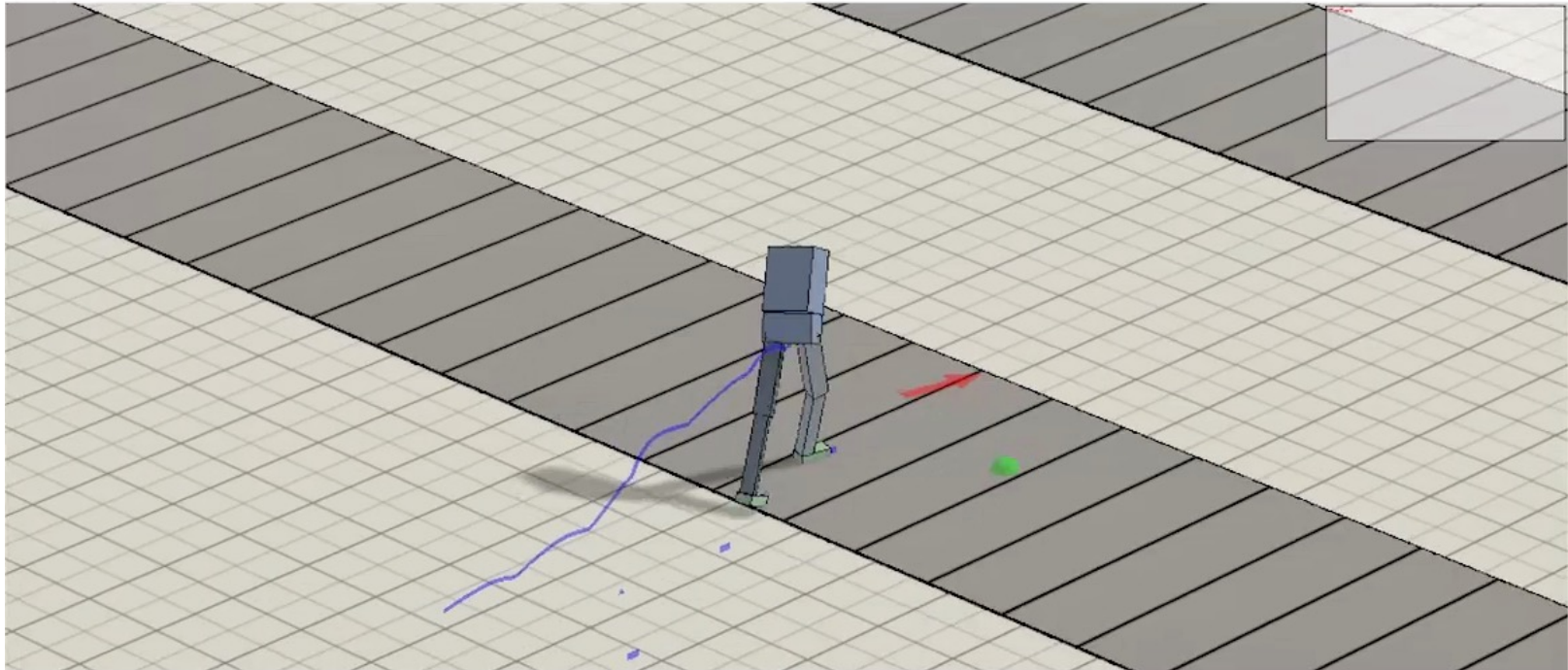
Anymal: Quadrupedal locomotion + getup (ETH Zurich)



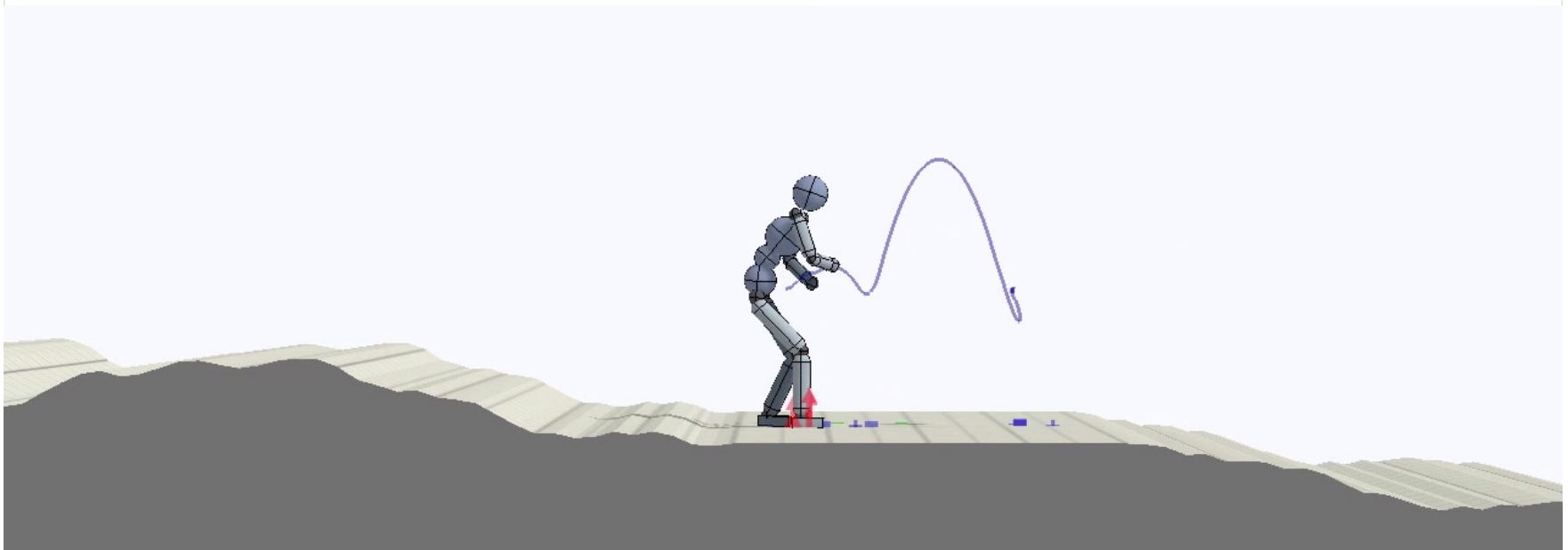
[Science Robotics, 2018]

[DeepLoco: SIGGRAPH 2017]

Walking on Conveyor Belts

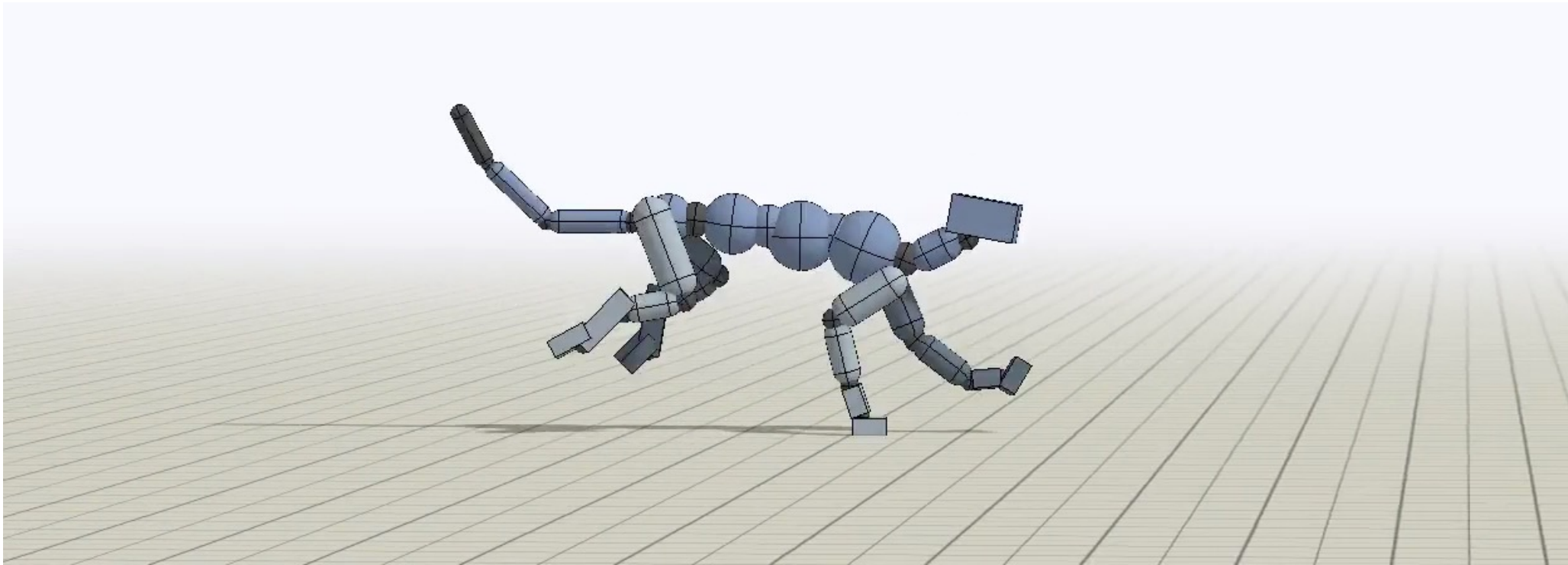


Retargeting



and retarget to different environments.

simulated lion



Physics-based Lion (UC Berkeley, UBC, Ziva Dynamics)



["DeepMimic", SIGGRAPH 2018]

Cassie: Bipedal Locomotion (UBC, U Oregon, Agility Robotics)



Why Deep RL? (continued)

End-to-end learning: performance benefits



Traditional approach



End to end learning

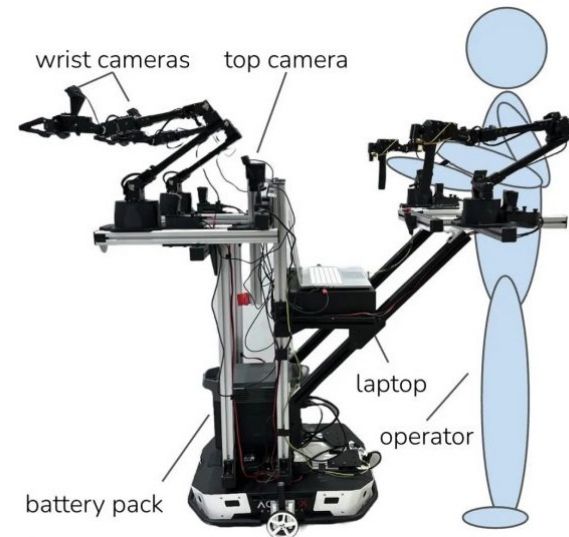


Self-optimized

[Chen and Huang, IV 2017]

“pixels to torques”

Good control from rich sensory streams
is often the limiting factor for robotics



[mobile ALOHA]

[<https://robotik.dfki-bremen.de/en/research/robot-systems/exoskelett-aktiv-ca.html>]

We may better understand how humans and animals learn

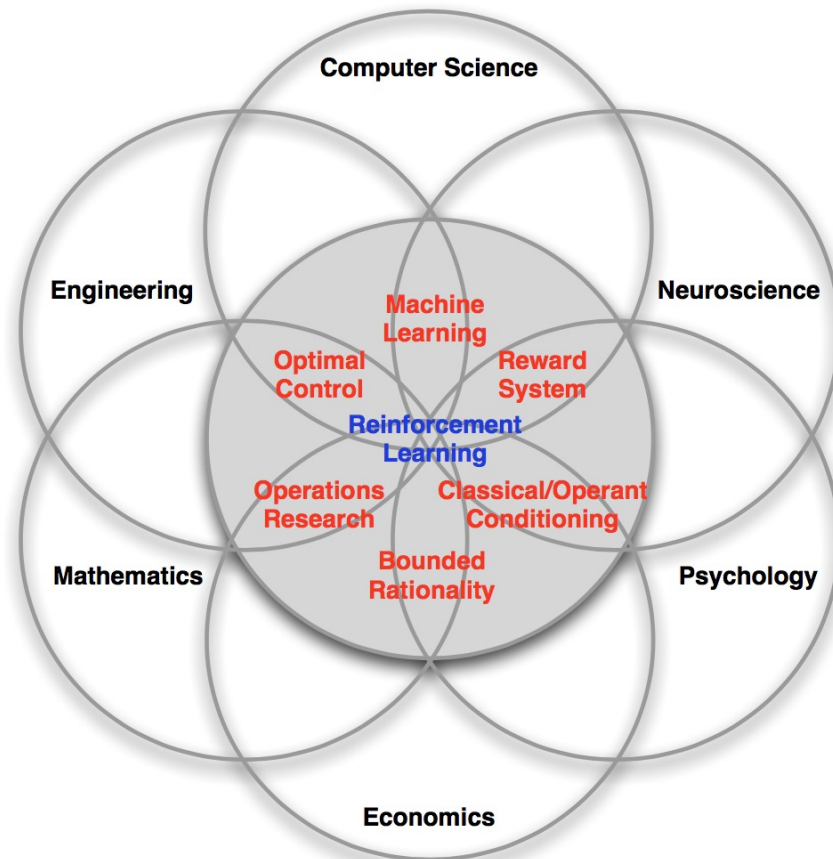


Follow the money

- Alphabet invests ~ \$2B in DeepMind
- Microsoft invests \$1B in OpenAI
- autonomous driving: Mobileye, Wayve, Waymo, Tesla, ...

Some Caveats

RL has a long history (by many names)



[Silver]

Generic Algorithm?

Best results often still have domain knowledge

- domain knowledge
 - defining: state, action, reward
 - hyperparameters
 - example expert data

but:

- no-free lunch: *inductive bias* needed for efficient learning
 - build on foundation models?

RL learning often has poor “sample efficiency”

- often far too slow to learn directly in the real world
 - e.g., thousands of years in simulation (OpenAI hand)
- learning requires good simulation models, so not truly “model free”
- many newborn animals can walk within minutes or hours: giraffe, horse foal, piglets, camels, zebra and more

but:

- compute is cheap
- transfer learning and generalization from related tasks: foundation models
- nature: has done much learning on an evolutionary time scale; this is a form of transfer learning



Alternatives to RL

- AI planning: given a known model of how action impacts world
 - e.g., online search through future states

- Imitation learning: just copy an expert!
 - reduces RL to supervised learning from an expert
 - collect data, learn policy
 - failure modes exist due to cumulative nature of errors
 - combining imitation + RL is a promising direction

$$\{(s_i, a_i)\}$$
$$\downarrow$$
$$a = \pi(s)$$



<https://umi-gripper.github.io/>

RL has limited data to learn from

[Yann LeCun 2016]

Y. LeCun

How Much Information is the Machine Given during Learning?

- ▶ **“Pure” Reinforcement Learning (cherry)**
 - ▶ The machine predicts a scalar reward given once in a while.
 - ▶ **A few bits for some samples**
- ▶ **Supervised Learning (icing)**
 - ▶ The machine predicts a category or a few numbers for each input
 - ▶ Predicting human-supplied data
 - ▶ **10→10,000 bits per sample**
- ▶ **Self-Supervised Learning (cake génoise)**
 - ▶ The machine predicts any part of its input for any observed part.
 - ▶ Predicts future frames in videos
 - ▶ **Millions of bits per sample**



RL can be difficult to apply in practice

Reinforcement Learning for Real Life

ICML 2019 Workshop

June 14, 2019, Long Beach, CA, USA

- Production systems
- Autonomous driving
- Business management
- Chemistry
- Computer Systems
- Energy
- Healthcare
- Robotics/manufacture

“Challenges of Real-World Reinforcement Learning”

[Dulac-Arnold et al.]

“ ... the research advances in RL are often hard to leverage in real-world systems due to a series of assumptions that are rarely satisfied in practice”

1. Training off-line from the fixed logs of an external behavior policy.
2. Learning on the real system from limited samples.
3. High-dimensional continuous state and action spaces.
4. Safety constraints that should never or at least rarely be violated.
5. Tasks that may be partially observable ...
6. Reward functions that are unspecified, multi-objective, or risk-sensitive.
7. System operators who desire explainable policies and actions.
8. Inference that must happen in real-time at the control frequency of the system.
9. Large and/or unknown delays in the system actuators, sensors, or rewards.

Reproducibility and Brittle Results

Deep Reinforcement Learning that Matters

**Peter Henderson^{1*}, Riashat Islam^{1,2*}, Philip Bachman²
Joelle Pineau¹, Doina Precup¹, David Meger¹**

¹ McGill University, Montreal, Canada

² Microsoft Maluuba, Montreal, Canada

In recent years, significant progress has been made in solving challenging problems across various domains using deep reinforcement learning (RL). Reproducing existing work and accurately judging the improvements offered by novel methods is vital to sustaining this progress. Unfortunately, reproducing results for state-of-the-art deep RL methods is seldom straightforward. In particular, non-determinism in standard benchmark environments, combined with variance intrinsic to the methods, can make reported results tough to interpret. Without significance metrics and tighter standardization of experimental reporting, it is difficult to determine whether improvements over the prior state-of-the-art are meaningful. In this paper, we investigate challenges posed by reproducibility, proper experimental techniques, and reporting procedures. We illustrate the variability in reported metrics and results when comparing against common baselines and suggest guidelines to make future results in deep RL more reproducible. We aim to spur discussion about how to ensure continued progress in the field by minimizing wasted effort stemming from results that are non-reproducible and easily misinterpreted.

[2017]

“Unfortunately, reproducing results for state-of-the-art deep RL methods is seldom straightforward.”

Course Administrivia

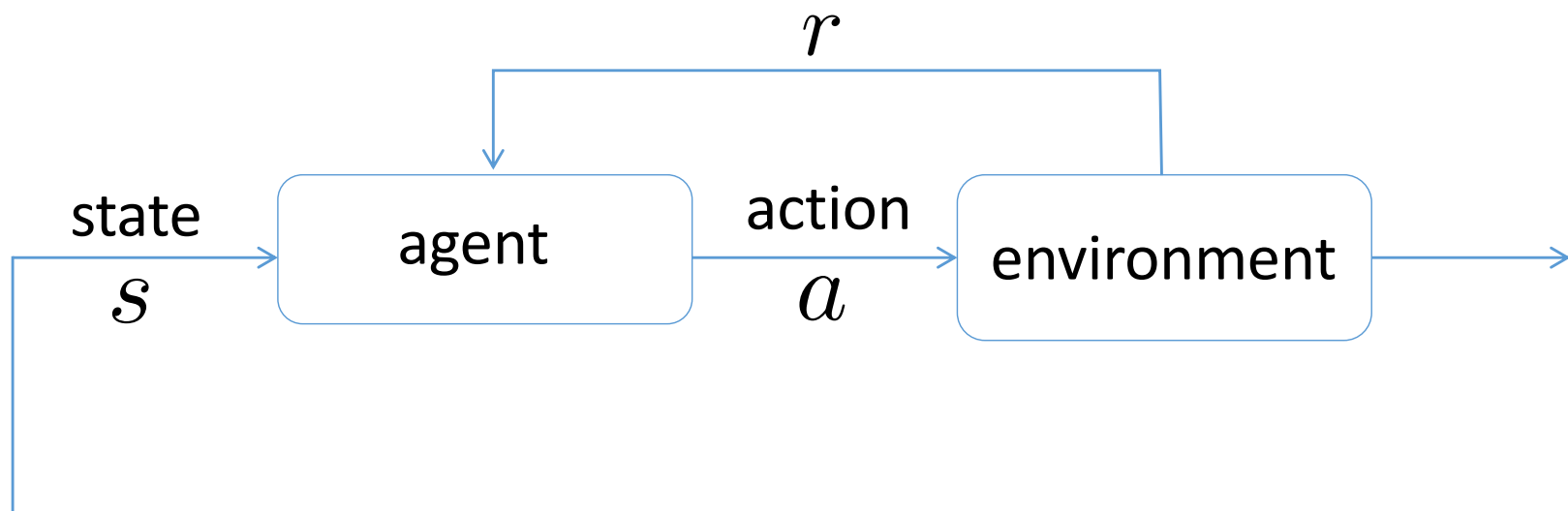
- registration / enrollment
- Piazza
- Assignments
- Project

Icebreaker – get to know other students

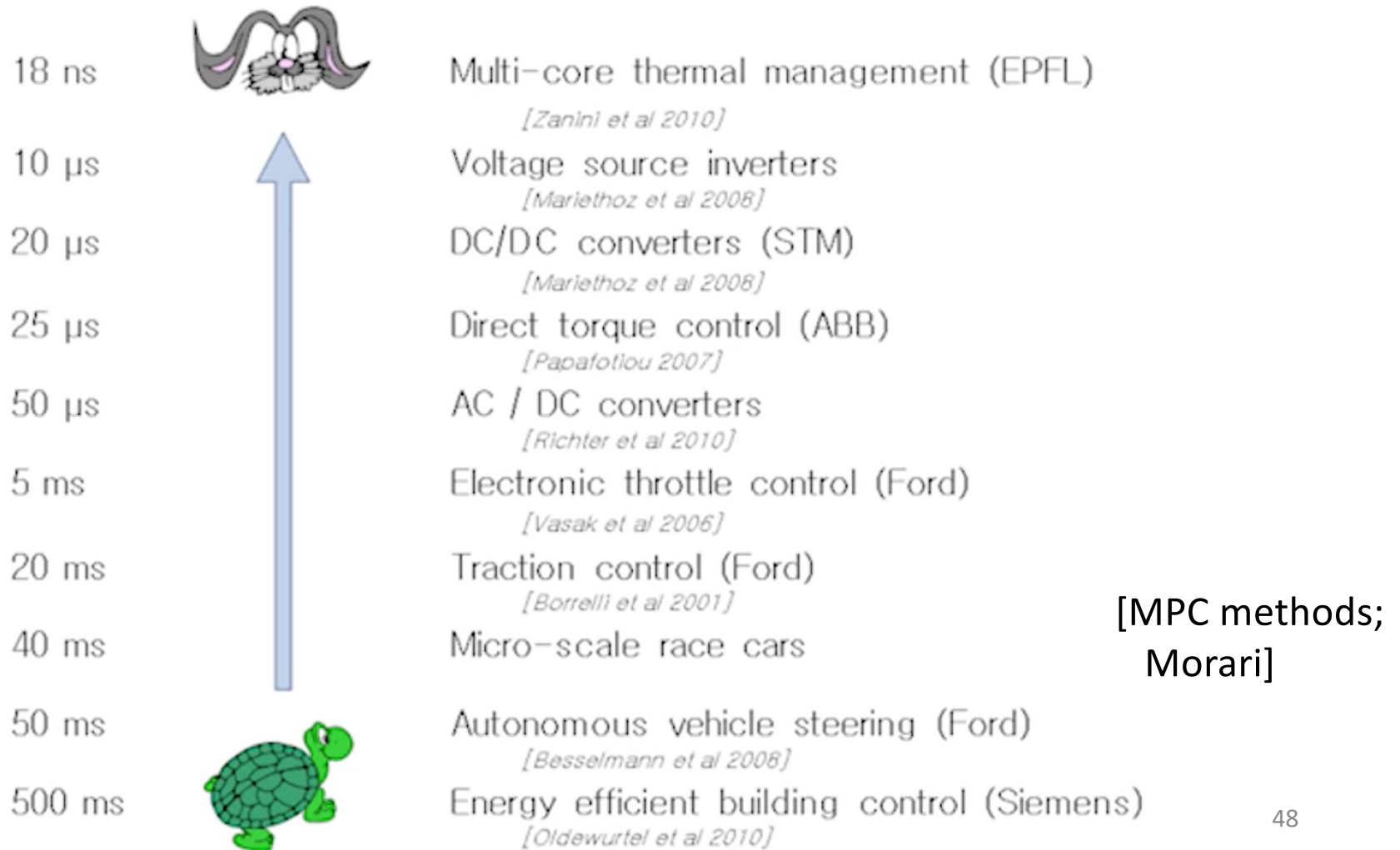
RL Basics

CS and Engineering views
(see handwritten notes)

Reinforcement Learning – basics



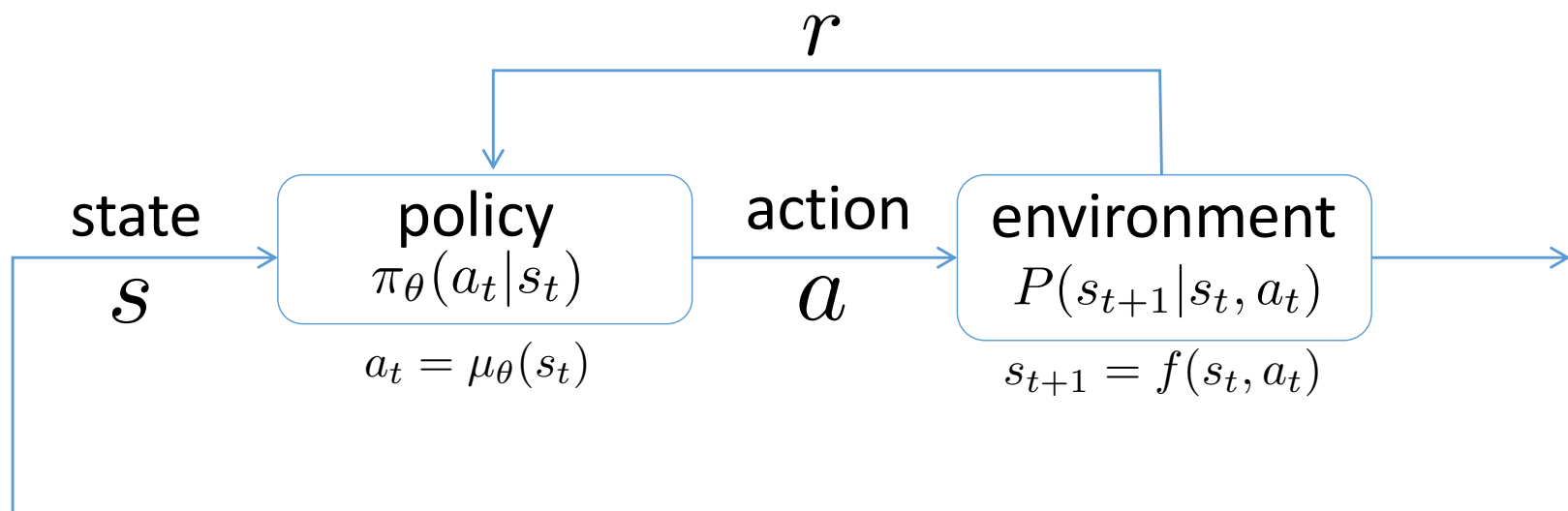
Decision Problems with various time scales



RL Terminology

- states, observations, actions, rewards
- environment / dynamics / state transition probability
- return, expected return
- imitation learning, behaviour cloning, NN distillation
- MDP, POMDP
- episodic task, discounting, stochastic policy

Markov Decision Process (MDP): $(S, A, P, R, S_0, \gamma)$



maximize $G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$

“return”, cumulative discounted rewards

Markov Property (notes)

Simplest Solution to Learning a Policy: “Behavior Cloning”

Observe what an expert does, and try to copy it:

$$\{(s_i, a_i)\}$$



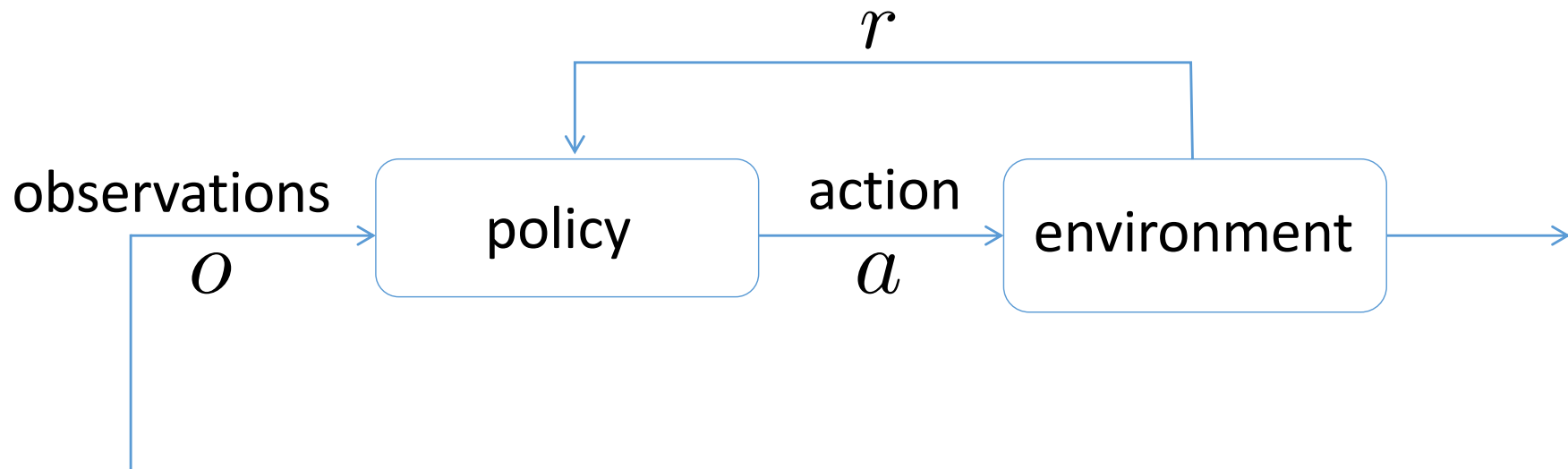
$$a = \pi(s)$$

“behaviour cloning”;

regresses actions onto states
(supervised learning)

- requires an expert
- learned policy is unaware of the actual task (!)
- suffers from compounding errors over time; can be mitigated by:
 - collecting data during perturbations
 - collecting expert data while following the student policy (“DAGGER” paper)
- transfer between two Deep-NN policies: “policy distillation” “policy cloning”

Partially Observable Markov Decision Process (POMDP)



commonly compensate by any of:

- using a state estimator, $s = f(o)$
- conditioning policy on history of observations
- conditioning policy on memory: recurrent neural network, LSTM, GRU

Common RL Assumptions

- *episodic* tasks
 - repeated interactions with the world e.g., games, trials
 - begins from a standard starting state (or distribution of states)
 - ends when reaching terminal state, or a fixed time T
- *discounting*: γ
 - avoid infinite rewards when $T = \infty$ (arguably a hack: could optimize mean reward)
 - uncertainty about the future:
“probability that the game will be allowed to continue after the current move”
 - useful regularizer for temporal difference learning
 - “effective horizon”: $H = \frac{1}{1 - \gamma}$
- *stochastic policies*: action probabilities $\pi(a|s)$ vs $a = \pi(s)$
 - “smooths” the learning, e.g., probability of playing a card, steering left, etc
 - provides exploratory actions (for some algorithms)
 - often switch to using a deterministic policy once learning is complete

More on discounting

$$G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

I always thought in reinforcement learning we try to conflate too many ideas into one number, γ , the discount factor. It serves as :

- (1) a technical tool to avoid computing with infinities*
- (2) a way to deal with a finite but uncertain lifetime, and*
- (3) a way to summarize our uncertainty about the future.*

In economics, there's also

- (4) a prediction about inflation rates or equivalently the net present value of a future financial instrument.*

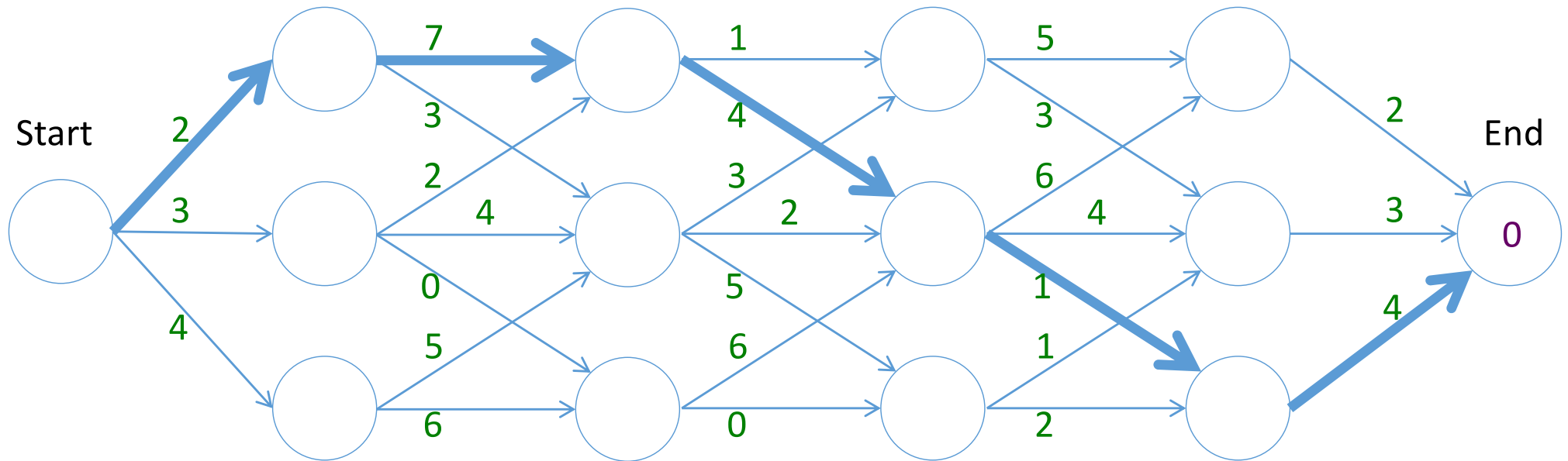
I think we'd be better off if we tried to model each of these things separately, instead of saying we'll have one hyperparameter and try to learn the best value for it.

-- Peter Norvig

RL (MDP) problem attributes

- state: {discrete, continuous, hybrid}
- action: {discrete, continuous, hybrid}
- initial state: {deterministic, stochastic}
- policy: {deterministic, stochastic}
- dynamics: {deterministic, stochastic}
- dynamics: {known, unknown (black-box), modeled via learning}
- state: {known, unknown (only have [stochastic] "observations")}
- time: {discrete, continuous}
- rewards: {deterministic, stochastic}
- task: {episodic, continuous}

Returns for a given sequence of decisions

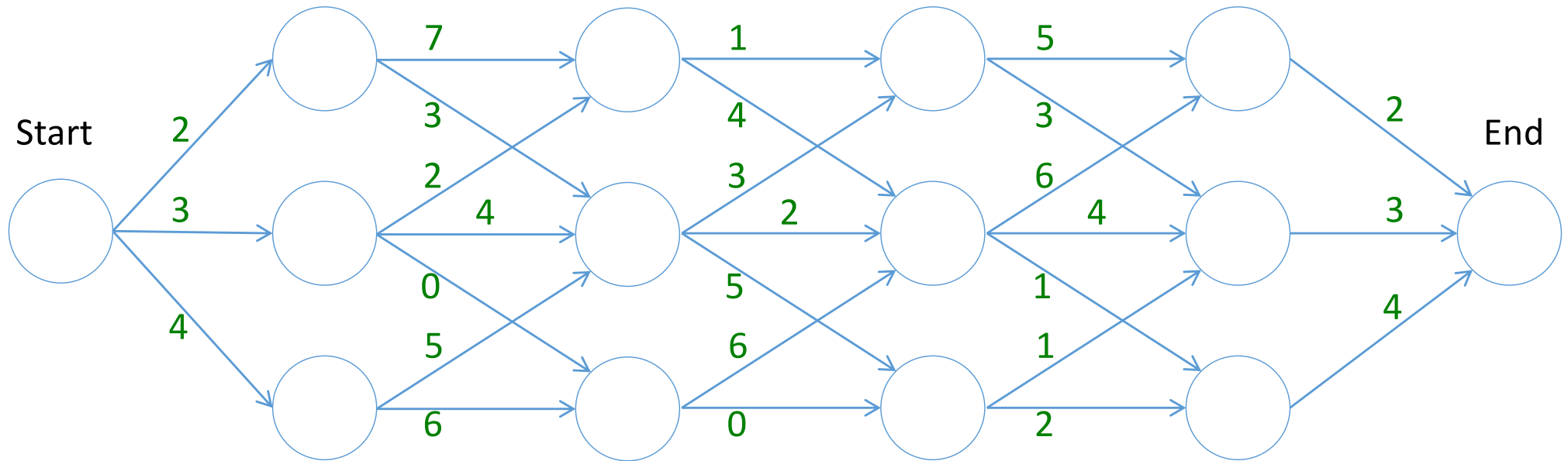


Assume $\gamma = 1$

Research ideas exist in all directions!

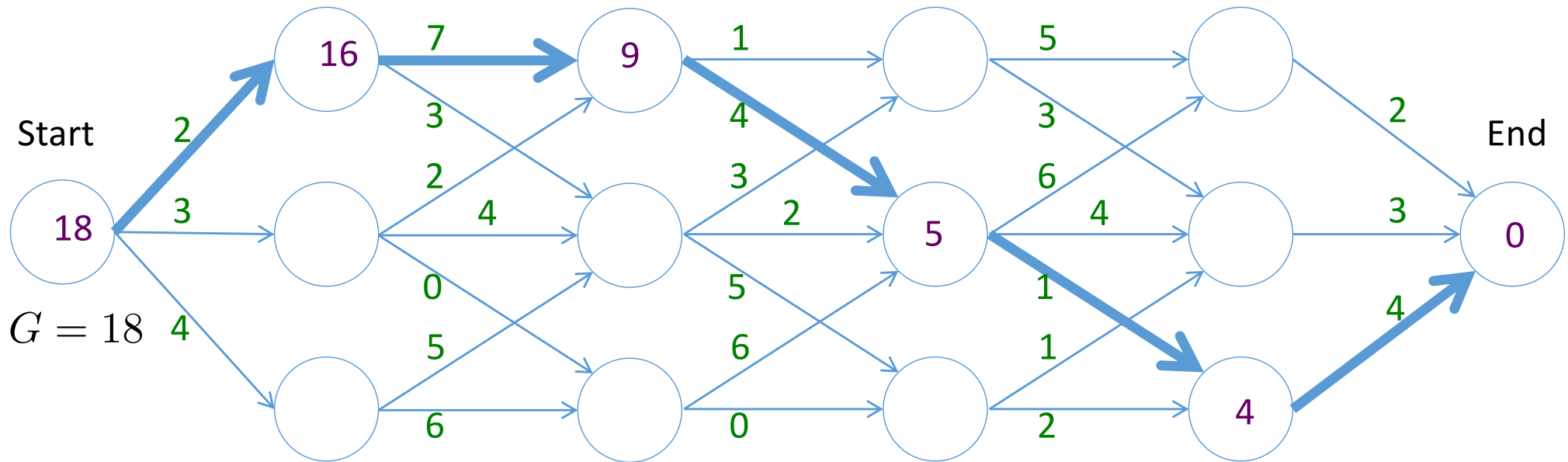
- how can we have humans provide rewards? (RL from human feedback)
- given a system and its behavior, can we infer the rewards? (inverse RL)
- can we learn without a fixed task? (continual/lifelong learning)
- how do we learn from similar tasks? (transfer learning; universal policies)
- should we learn easy tasks, then difficult tasks? (learning curricula)
- how can we leverage expert (or non-expert) data? (imitation methods; offline RL)
- can we learn how to learn? (meta-learning)
- how can we leverage known-or-modeled dynamics? (model-based RL)
- how can we meaningfully evaluate RL results? (“Deep RL that matters”)
- what is the best way to debug RL algorithms?

A Simple Discrete Deterministic Example



Returns for a given sequence of decisions

“trajectory” $\mathcal{T}$: (s,a,r, ... s,a,r)



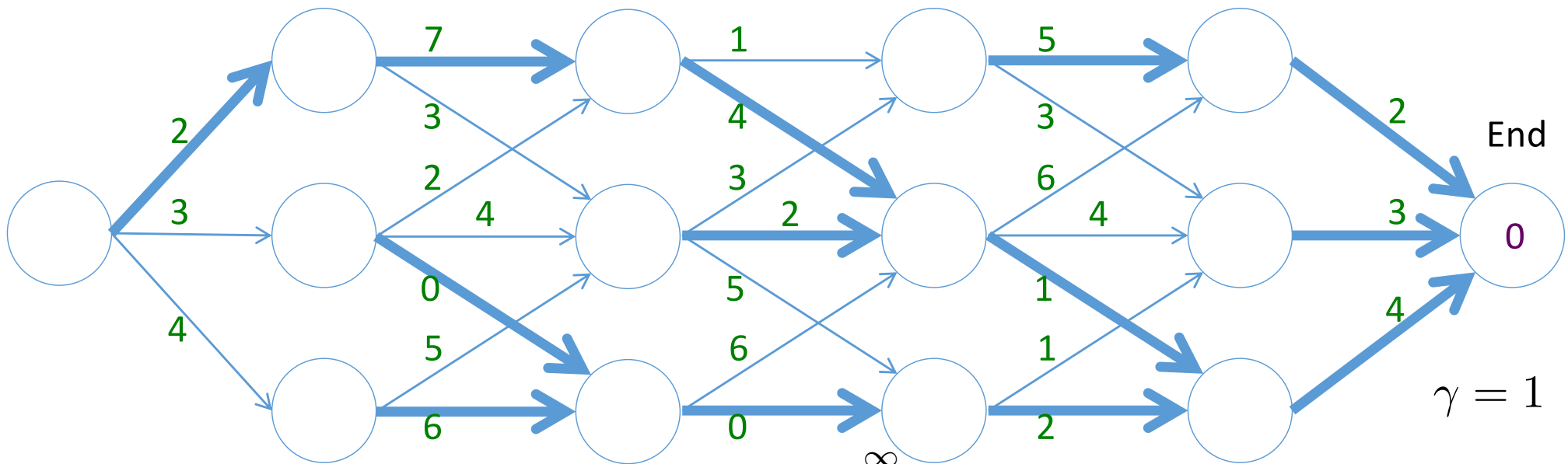
$$G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad \text{Above we have } \gamma = 1$$

$$\gamma = 0.9 : G(\text{start}) = 2 + (0.9)7 + (0.9)^2 4 + (0.9)^3 1 + (0.9)^4 4 \quad 60$$

Returns for a given policy π

“policy evaluation”

Assume that we begin in a random state, for this example, “exploring start” and therefore we care about the returns from all states.

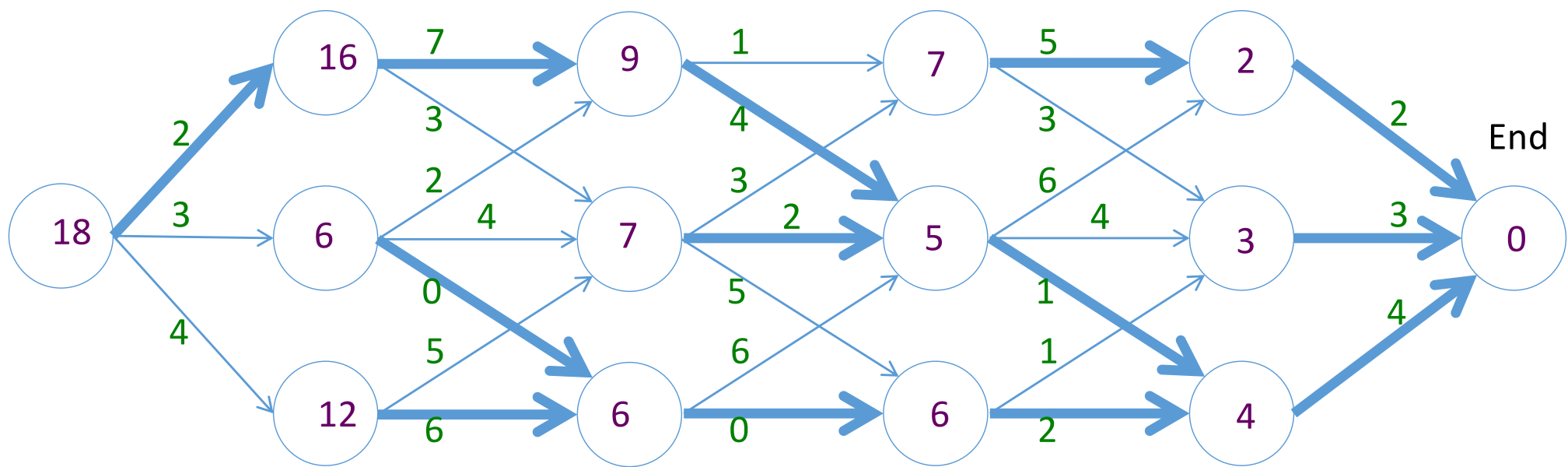


actual returns $G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$

expected returns $V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$

for deterministic rewards, policy, dynamics
actual returns = expected returns ⁶¹

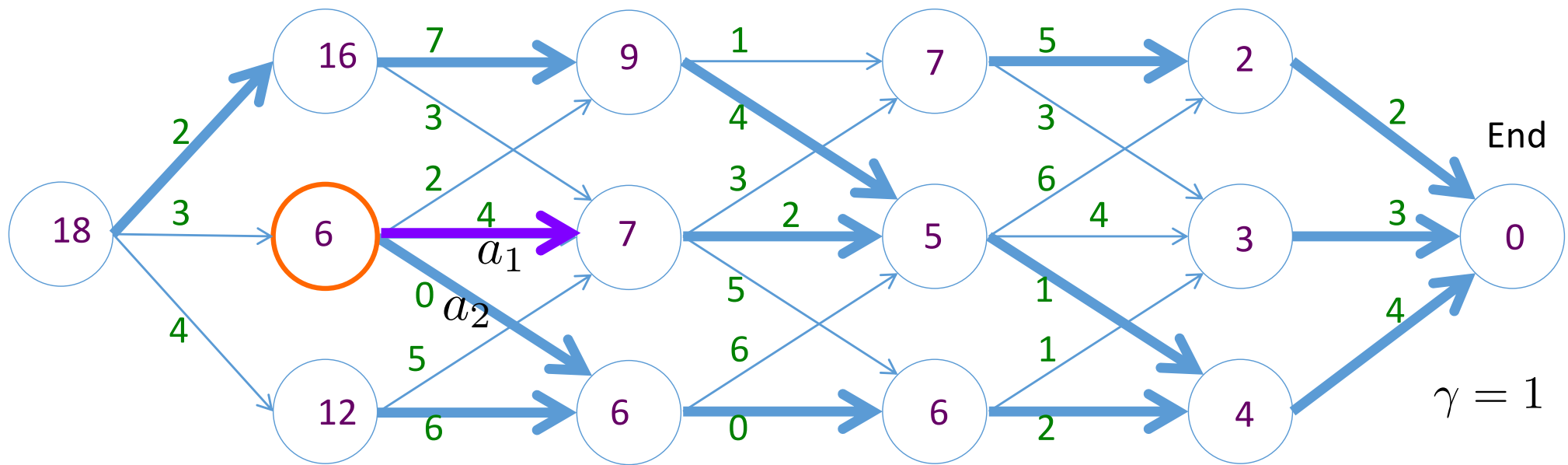
Returns for a given policy π



$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

“policy improvement”

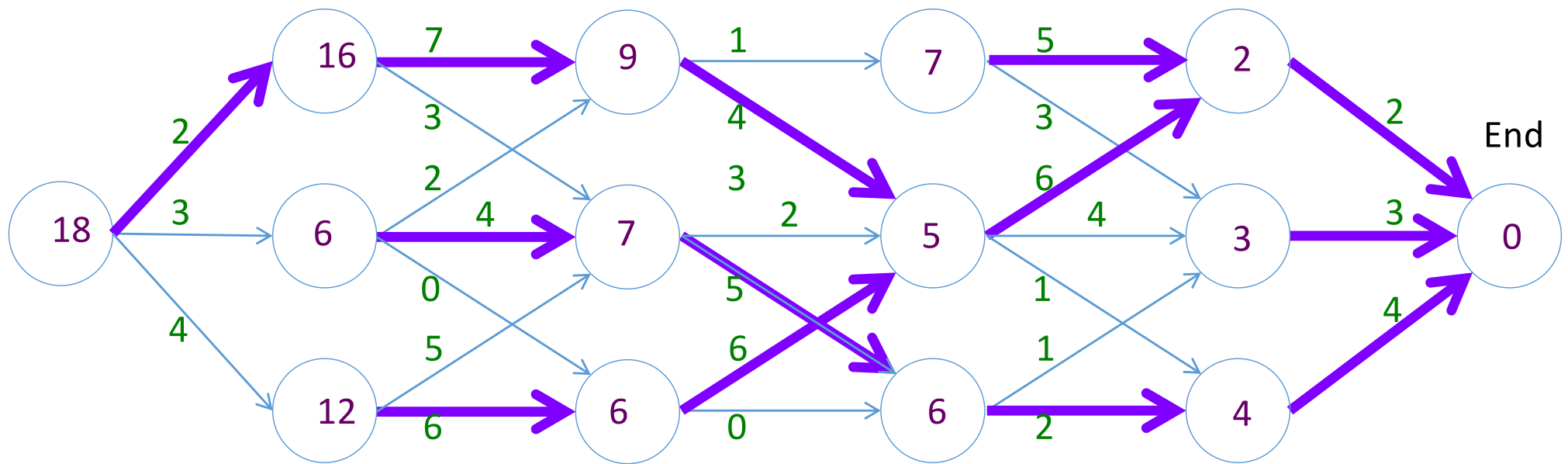
Given the known returns, now improve the policy



$$\pi(s) \leftarrow \arg \max_a \sum_{s', r} [r(s, a) + \gamma V(s')]$$

a_1	$4 + 7 = 11$	← best action
a_2	$0 + 6 = 6$	

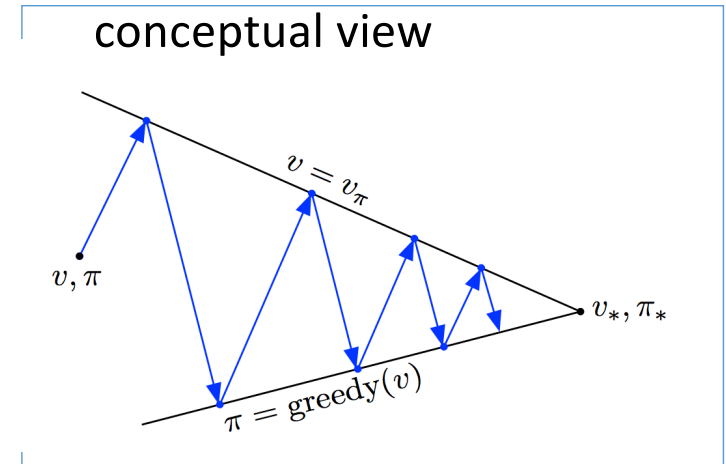
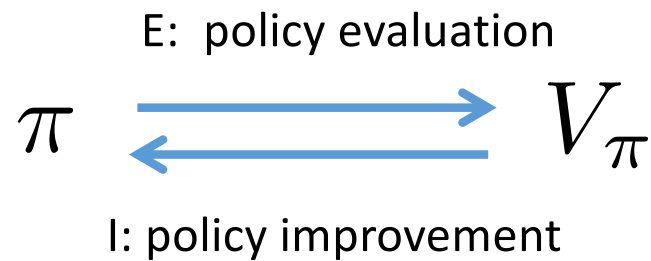
Final improved policy



Note: the value function is now out of date

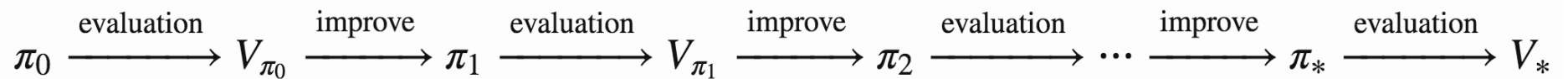
Terminology

- *Generalized Policy Iteration*



optimal policy,
optimal value fn

V_*, π_*



Terminology

- *state value function* $V(s)$:
 $V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$
 - **expected** returns for a given state, under a given policy “how good is a state?”
 - for our deterministic example,
 $V_{\pi}(s) = G_{\pi}(s)$
- *value iteration, with bootstrapping*:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

$$G_t = r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} + \dots)$$

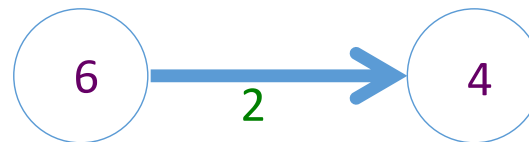
$$G_t = r_{t+1} + \gamma G_{t+1}$$

similarly for the value functions

for each iteration k

for each state s

$$V_{k+1}(s) = r + \gamma V_k(s')$$

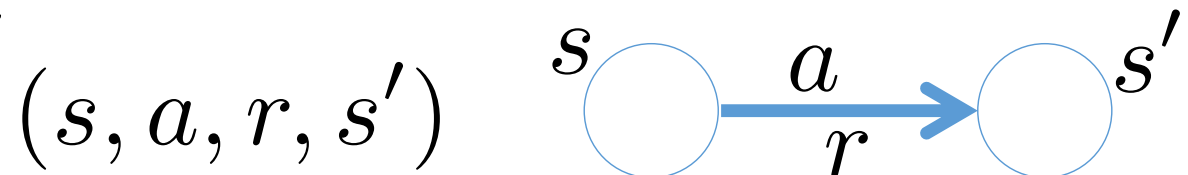


- iterative value estimates depend on previous value estimates (!)
- *prioritized sweeping*: order updates based on size of expected change in V

Terminology

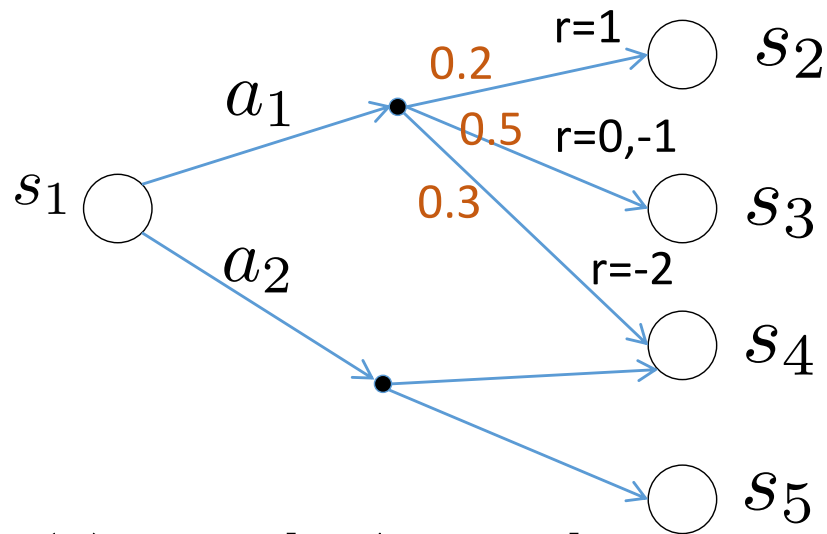
- *discount factor* $0 \leq \gamma \leq 1$
 - allows for continuing episodes, i.e., infinite length, which would otherwise produce a potentially infinite return
 - probability of episode continuing at any given time
 - preference for current rewards over future rewards, given that there may be more uncertainty about the future rewards

- *experience tuples*



- from state s , we took action a , and received reward r and ended in state s'

Stochastic Environments



$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

$$V_{\pi,k+1}(s) = \sum_{s',r} p(s',r|s,a)[r + \gamma V_{\pi,k}(s')]$$

dynamics function: $p(s', r | s_1, a_1)$
“state transition model”

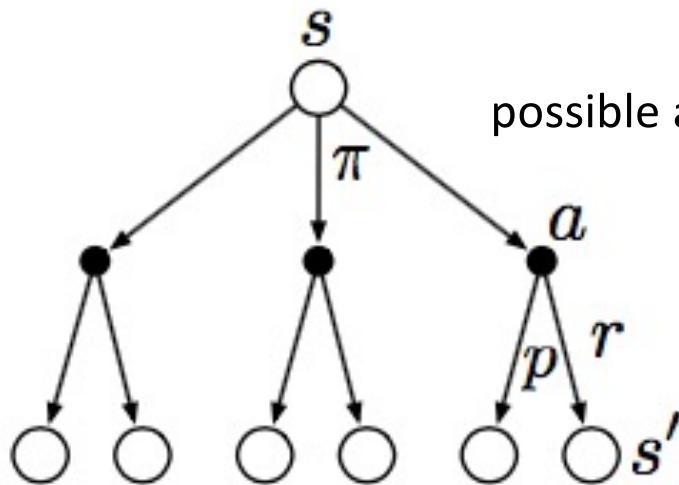
when in state s_1 and taking action a_1
what is the probability of ending up
in state s' and receiving reward r

$p(s', r s_1, a_1)$		next state	reward
stochastic reward	0.2	s_2	1
	0.4	s_3	0
	0.1	s_3	-1
	0.3	s_4	-2

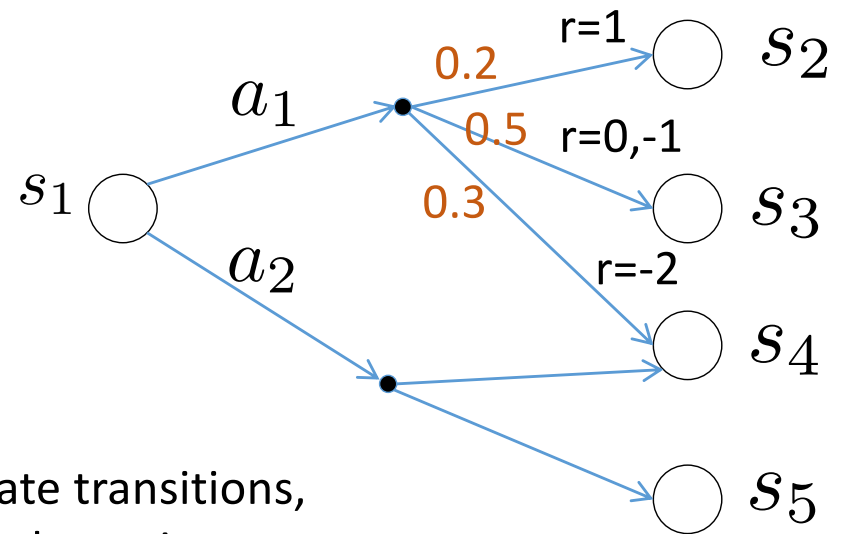
↑ sums to 1

The value function provides the **expected returns**

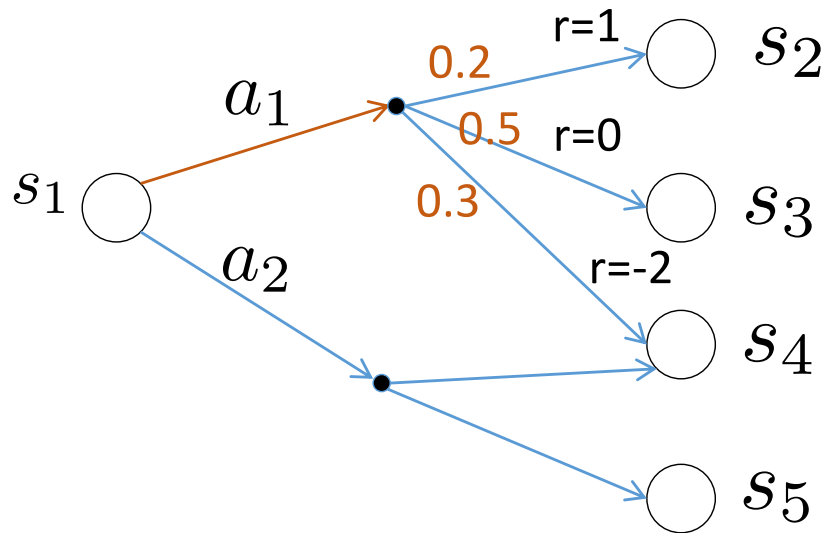
Common “backup diagram”



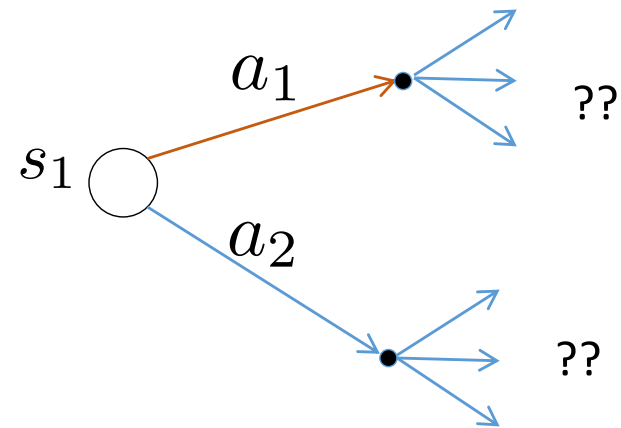
[Sutton, p81]



Model-based



Model-free



$$V_{\pi,k+1}(s) = \sum_{s',r} p(s',r|s,a)[r + \gamma V_{\pi,k}(s')]$$

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

model-based: all state-transitions and rewards are known in advance, follow some known distribution

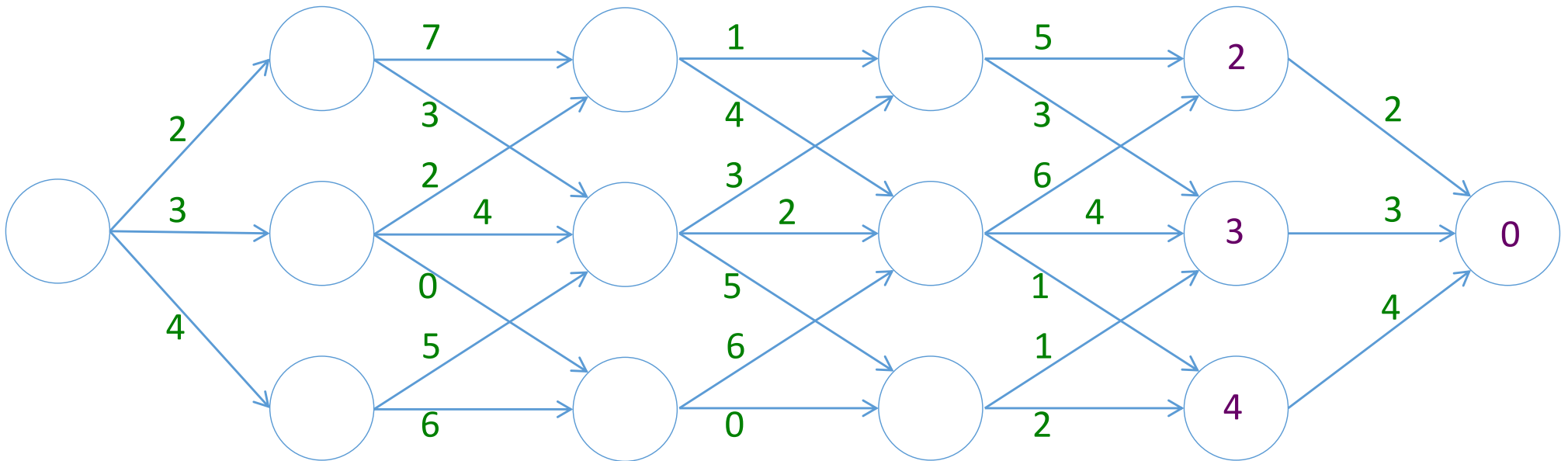
model-free: state-transitions and rewards are not known [therefore need to discover these through experience!]

most common: model-free; More advanced: model-based methods learn an approximate models for $p()$

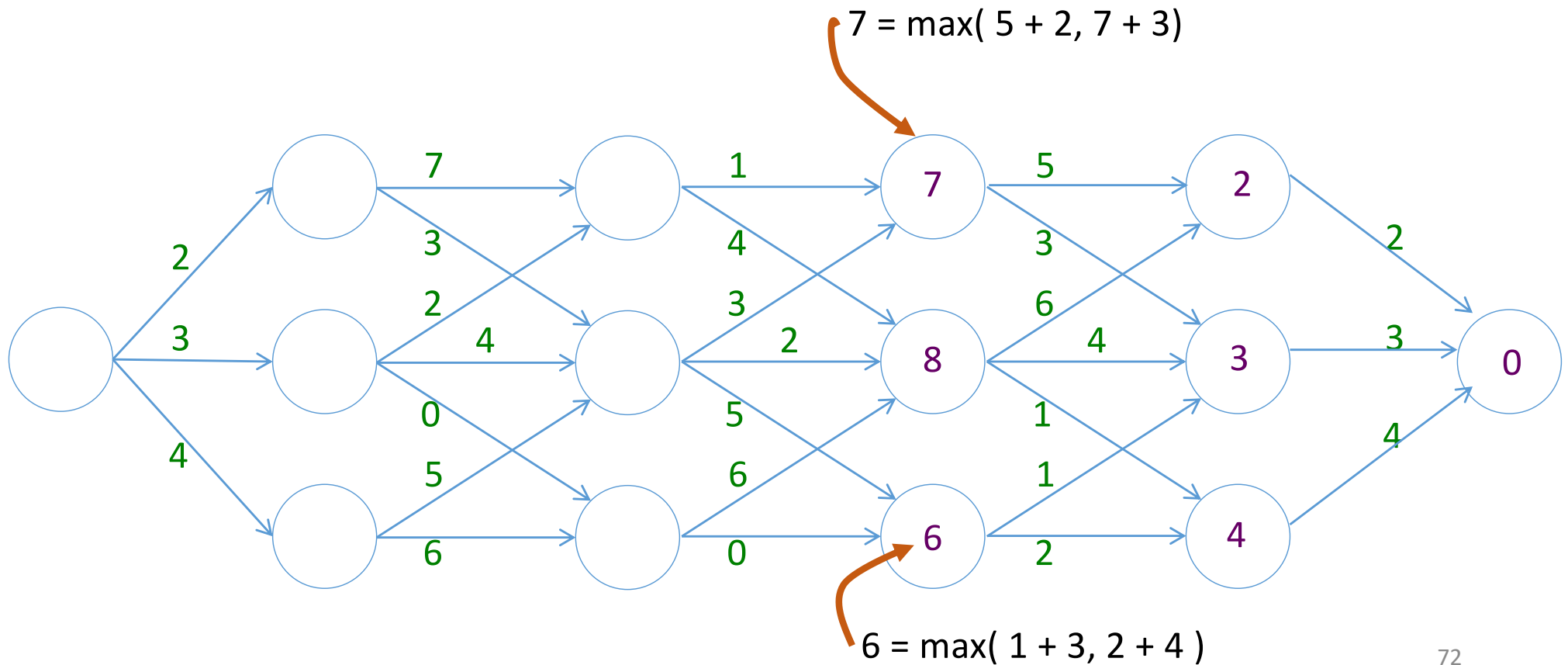
Value Iteration:

Find the best returns for every state; implicit policy

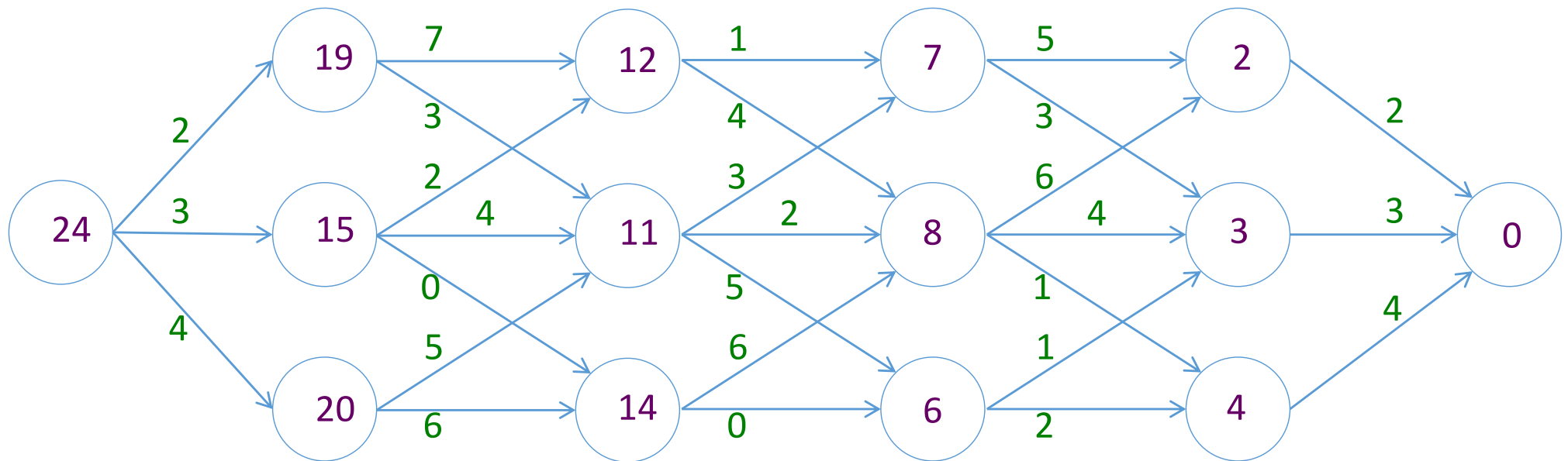
("Dynamic Programming" because the model is known)



“Bellman Backups”



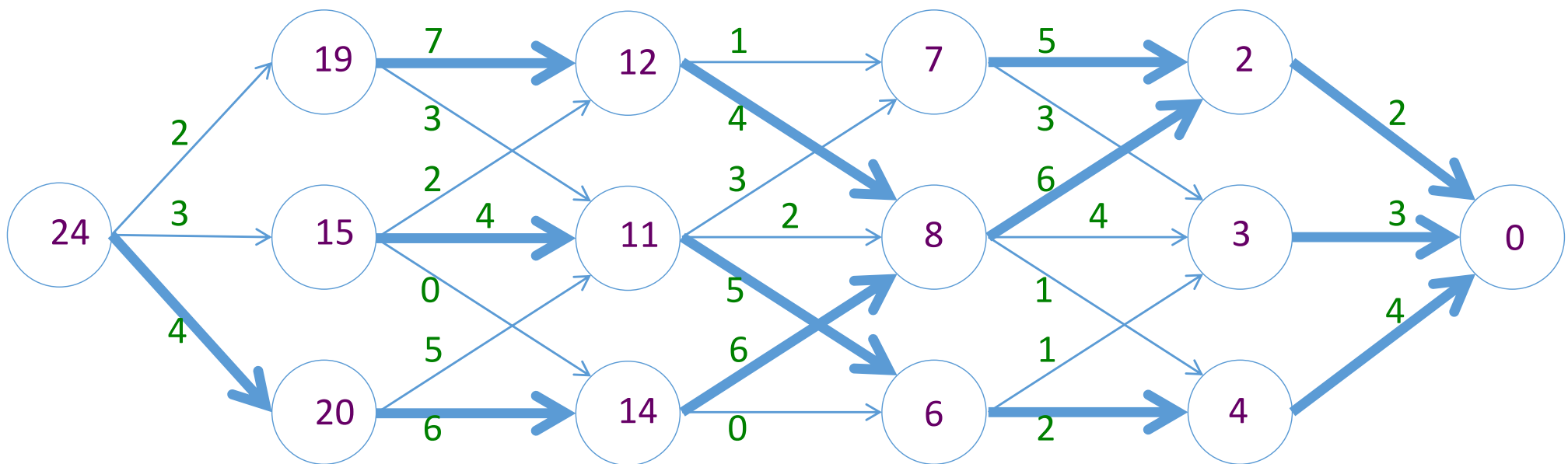
Optimal Value Function



Optimal Policy (implicit in optimal value fn)

value fn update:
$$V_{k+1}(s) = \max_a (r + \gamma V_k(s'))$$

related implicit policy:
$$\pi_{k+1}(s) = \arg \max_a (r + \gamma V_k(s'))$$



More Definitions

- *Bellman backup*: a local iterative improvement of the value function
 - deterministic dynamics:

$$V_{k+1}(s) = \max_a (r + \gamma V_k(s'))$$

- stochastic dynamics:

$$V_{k+1}(s) = \max_a \left(\sum_{s', r} p(s', r | s, a) [r(s, a) + \gamma V_k(s')] \right)$$

Policy Iteration [Dynamic Programming]

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

[synchronous vs in-place updates?]

Loop:

[linear system: direct or iterative soln]

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow true$

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow false$

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

[Sutton, p80]

Value Iteration [Dynamic Programming]

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$ 
| Loop for each  $s \in \mathcal{S}$ :
|    $v \leftarrow V(s)$ 
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$    Bellman backup
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

“Value iteration effectively combines, in each of its sweeps, one sweep of policy evaluation and one sweep of policy improvement.”

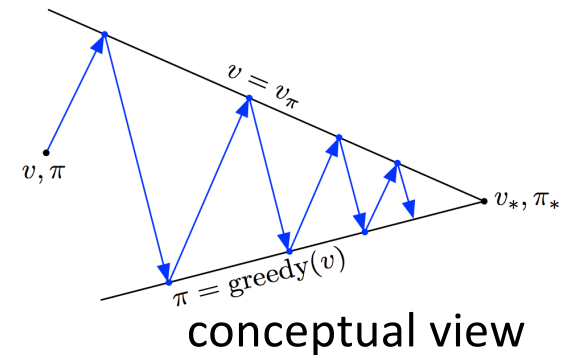
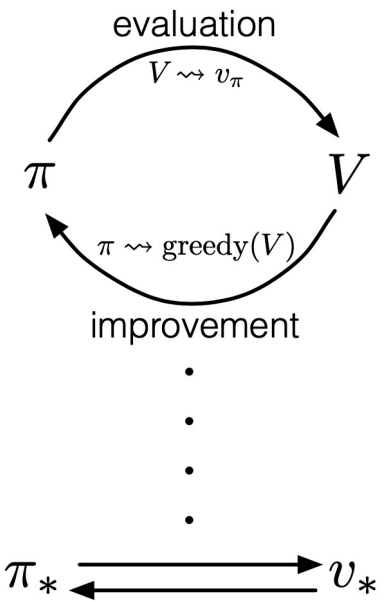
[Sutton, p83]

Asynchronous Dynamic Programming

- previous methods require full sweeps of all states, which is impossible for interesting games, e.g., Backgammon, Chess, Go
- asynchronous DP algorithms update the values of states in any order
- useful in online play, e.g., focus on relevant future game states

“Almost all reinforcement learning methods are well described as GPI. That is, all have identifiable policies and value functions, with the policy always being improved with respect to the value function and the value function always being driven toward the value function for the policy” [Sutton, p. 86]

Note: RL algorithms will typically be optimizing the value function and policy together, instead of strict alternation.



Remarks

- in general, need repeated sweeps through all states (our case: one sweep)
- dynamic programming: all states, actions are visited; dynamics known
 - discrete states and actions: planning on a graph; Viterbi algorithm
 - continuous states and actions
 - linear system, quadratic cost function: Linear Quadratic Regulator (known optimal solution)
 - more arbitrary dynamics and costs: Hamilton-Jacobi solvers (for low-D systems)
- constructing the policy requires knowing the dynamics model, $p()$.

But in general “model free” RL, we don’t know the model,

Dynamic Programming in other forms

- solving MDPs using Linear Programming
- solving continuous-state MDPs using Hamilton-Jacobi-Bellman Equations

Linear Programming

Linear programs are problems that can be expressed in **canonical form** as

Find a vector	$\mathbf{x}$
that maximizes	$\mathbf{c}^T \mathbf{x}$
subject to	$A\mathbf{x} \leq \mathbf{b}$
and	$\mathbf{x} \geq \mathbf{0}.$

[Wikipedia]

Solving an MDP using Linear Programming

$$\begin{aligned} & \underset{V}{\text{minimize}} && \sum_s V(s) \\ & \text{subject to} && V(s) \geq R(s) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V(s'), \quad \forall a \in \mathcal{A}, s \in \mathcal{S} \end{aligned}$$

Typically, best specialized MDP algorithms (e.g. modified policy iteration) are faster than general LP algorithms, but the LP formulation provides a number of connections to other methods, and has also been the basis for much work in approximate large-scale MDP solutions (e.g., de Farias and Van Roy, 2003)

[Kolter 2016]

Hamilton-Jacobi-Bellman Equation

Solve an n-dimensional PDE that defines the value function in the state space.

$$-V_t(t, x) = \min_u (c(t, x, u) + V_x(t, x)f(t, x, u))$$

Boundary Condition: $V(t_f, x(t_f)) = \phi(x(t_f))$

Numerically solve backwards in time for all (t,x) to obtain the closed-loop optimal control policy:

$$u^*(t, x) = \arg \min_u (c(t, x, u) + V_x(t, x)f(t, x, u))$$

[<https://www.slideshare.net/HarukiNishimura/stochastic-control-and-information-theoretic-dualities-complete-version>]

Hamilton-Jacobi-Bellman equations

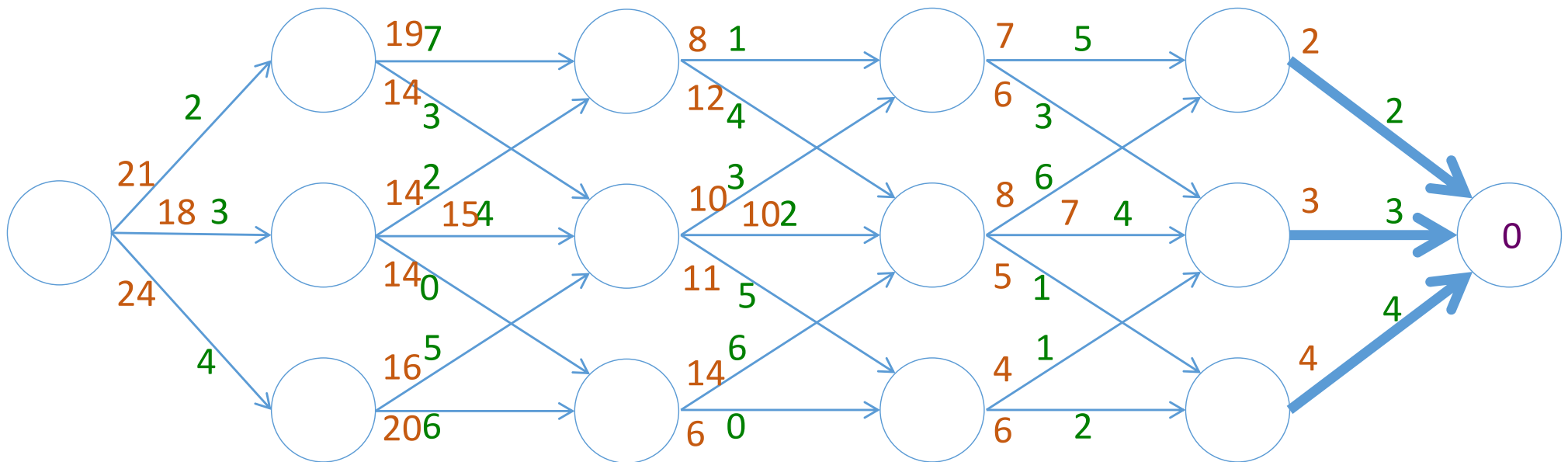
- continuous-state, continuous-action problems
- a non-linear PDE (partial differential equation) in the value function, whose solution is the value function itself
- Eikonal equation (optics!) is a particular case of HJB equations
- numerical solutions, often using level-set methods
 - solution evolves backwards in time
 - compute level sets in the value function
- feasible in low-D spaces, e.g., $\leq 7D$
- see CS Prof Ian Mitchell's work

RL algorithm attributes

- state space: discrete vs continuous
- action space: discrete vs continuous
- environment: stochastic vs deterministic
- task: episodic vs continuing
- policy: stochastic vs deterministic
- learning: model-free vs model-based
- learning: with-critic vs without-critic
- learning: on-policy vs off-policy
- learning: tabula rasa vs with demonstrations
- learning: simulation vs real-world

Learning with State-Action Values: $Q(s,a)$

(store cost-to-go on the edges)



$$Q_{k+1}(s, a) = r + \gamma \max_{a'} (Q_k(s', a'))$$

State-Action Value function: $Q(s,a)$ (equivalently called “Action Value” function)

- caches the expected returns of each possible action, when in a given state

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, a_t = a]$$

- small problems with discrete states, discrete actions: store as a table
- for continuous state space and N discrete actions
think of learning N value functions, one for each action: $Q_{\pi}^N(s)$
- policy then becomes trivial:

$$\pi(s) = \arg \max_a (Q(s, a))$$

- Bellman backup
 - deterministic case:

$$Q_{k+1}(s, a) = r + \gamma \max_{a'} (Q_k(s', a'))$$

Deep RL: key building blocks

